

Laboratorio N°1

Unidad Curricular: Tecnologías de Microprocesamiento

01/09/2026

Rosella Merello

Universidad Tecnológica del Uruguay
ITRSO
Paysandú, Uruguay
rosella.morello@estudiantes.utec.edu.uy

Diego Vázquez

Universidad Tecnológica del Uruguay
ITRSO
Paysandú, Uruguay
diego.vazquez@estudiantes.utec.edu.uy

Luis Nuñez

Universidad Tecnológica del Uruguay
ITRSO
Fray Bentos, Uruguay
luis.nunez@estudiantes.utec.edu.uy

Abstract— En este laboratorio se implementaron cuatro aplicaciones con el microcontrolador ATmega328P programado en lenguaje ensamblador: una lavadora automática, un contador decimal con display de siete segmentos, ocho secuencias de LEDs y una unidad aritmético-lógica (ALU) de 4 bits. Los sistemas fueron verificados mediante simulación y montaje físico con Arduino Uno.

Keywords— ATmega328P, GPIO, ensamblador AVR, máquina de estados finitos, retardos por software

1. RESUMEN

En este laboratorio se implementaron cuatro aplicaciones utilizando el microcontrolador ATmega328P y lenguaje ensamblador: una lavadora automática, un contador decimal con display de siete segmentos, ocho secuencias de LEDs y una unidad aritmético-lógica (ALU) de 4 bits. Los ejercicios permitieron aplicar el manejo de entradas y salidas digitales, retardos y operaciones aritméticas y lógicas, verificando su funcionamiento mediante simulación y montaje físico.

2. OBJETIVOS

A. Objetivo general

Desarrollar e implementar aplicaciones de control digital utilizando los puertos GPIO del microcontrolador ATmega328P mediante programación en lenguaje ensamblador AVR, aplicando técnicas de configuración, lectura y escritura de puertos, control de secuencias y verificación mediante simulación e implementación física.

B. Objetivos específicos

- Configurar y utilizar los puertos GPIO del microcontrolador ATmega328P como entradas y salidas digitales de acuerdo con los requerimientos de cada problema.
- Desarrollar programas en lenguaje ensamblador AVR aplicando instrucciones de manejo de bits, comparaciones, saltos, retardos y control de dispositivos digitales.

- Verificar el funcionamiento de las soluciones desarrolladas mediante simulación y su posterior implementación física, identificando y corrigiendo los errores encontrados durante el proceso.

3. MATERIALES

Para la realización de esta práctica de laboratorio fueron necesarios los siguientes elementos:

- Placa compatible con Arduino Uno con microcontrolador ATmega328P.
- Protoboard.
- LEDs.
- Resistencias limitadoras para LEDs.
- Pulsadores.
- Cables de conexión (*jumpers*).
- Cable USB.
- Computadora para programación y simulación.
- Microchip/Atmel Studio.
- PICSIMLAB.
- AVRDUDE.

4. MARCO TEÓRICO

Antes de detallar el procedimiento llevado a cabo, es necesario aclarar distintos conceptos importantes para la comprensión total de la práctica.

A. GPIO y registros de puerto en el ATmega328P

Cada puerto del ATmega328P (B, C, D) se controla mediante tres registros de 8 bits: DDRx define la dirección de cada pin (1=salida, 0=entrada), PORTx establece el nivel de salida o, cuando el pin está configurado como entrada, habilita la resistencia de pull-up interna, y PINx permite leer el nivel lógico presente en el pin. Esta práctica utiliza los tres puertos: B y C como salidas para los LEDs indicadores, y D de forma mixta, con entradas para pulsadores y sensores y salidas para el control del motor.

B. Antirrebote (debounce) por software

Los pulsadores mecánicos generan múltiples transiciones espurias entre 0 y 1 durante los primeros milisegundos de una pulsación o liberación, producto del rebote físico de los contactos. La técnica de antirrebote por software consiste en, al detectar un cambio de nivel, esperar un breve retardo (en esta práctica, 25 ms) y volver a leer el pin para confirmar que el cambio es estable antes de considerarlo un evento válido.

C. Retardos por software

En ausencia de temporizadores por hardware, los retardos se generan mediante lazos de instrucciones cuyo número de ciclos de reloj es conocido. Para el ATmega328P operando a 16 MHz, un retardo de 1 ms equivale a 16000 ciclos de reloj; combinando un lazo interno y uno externo (dec/brne) puede aproximarse este valor con un error menor al 0.1%, y a partir de esa base construirse retardos de mayor duración por composición.

D. Máquina de estados finitos (FSM)

El comportamiento del sistema se modela como una FSM: el sistema se encuentra en todo momento en uno de un conjunto finito de estados (IDLE, LAVADO, CENTRIFUGADO, SECADO, FIN), y transiciona entre ellos en respuesta a eventos de entrada (pulsadores, sensores) o a la finalización de un temporizador. Esta abstracción permite estructurar el programa de forma modular y verificar su lógica de forma independiente del detalle de implementación de cada estado.

E. Unidades Aritmético-Lógicas (ALU)

Una ALU es el bloque combinacional encargado de ejecutar operaciones aritméticas y lógicas sobre uno o más operandos binarios, seleccionando la operación mediante una señal de control. En este laboratorio se emula en software, mediante ensamblador AVR, la tabla de verdad de una ALU de 4 bits con ocho operaciones posibles, seleccionadas por una entrada de 3 bits (S2 S1 S0).

F. Registros y banderas de estado en AVR

El ATmega328P dispone de 32 registros de trabajo de 8 bits (r0-r31) y un registro de estado SREG que refleja el resultado de operaciones aritméticas y lógicas mediante banderas individuales (Carry, Zero, Negative, entre otras). Dado que en esta práctica los operandos son de 4 bits y las operaciones se ejecutan internamente sobre registros de 8 bits, las banderas C, N y Z se calculan explícitamente por software a partir del resultado, en lugar de leerse directamente del SREG, de modo de ajustarlas a la semántica de 4 bits requerida.

5. PROCEDIMIENTO

A. Lavadora Automática

1) Diseño de la máquina de estados y mapa de pines

Antes de escribir código se definió el comportamiento esperado del sistema y se estableció el mapa de pines que se utilizaría, resumido en la Tabla I. Los pulsadores de inicio y selección de carga se configuraron con resistencia de pull-up (interna y externa según la etapa de verificación) por ser activos en bajo, mientras que los sensores de puerta y agua se consideraron activos en alto.

TABLE I. MAPA DE PINES DEL ATMEGA328P

Puerto/Pin	Función	Dirección
PB0	LED - Listo para lavar	Salida
PB1	LED - Proceso de lavado	Salida
PB2	LED - Proceso de centrifugado	Salida
PB3	LED - Proceso de secado	Salida
PB4	LED - Fin del proceso	Salida
PC0	LED - Carga ligera	Salida
PC1	LED - Carga media	Salida
PC2	LED - Carga pesada	Salida
PD2	Pulsador de inicio (pull-up, activo en bajo)	Entrada
PD3	Pulsador de selección de carga (pull-up, activo en bajo)	Entrada
PD4	Sensor de puerta cerrada (activo en alto)	Entrada
PD5	Sensor de llenado de agua (activo en alto)	Entrada
PD6	Motor - encendido/apagado	Salida
PD7	Motor - dirección de giro (1=derecha, 0=izquierda)	Salida

La Tabla II describe los cinco estados definidos para la máquina y sus condiciones de transición. El diagrama de flujo de estados es el siguiente:

IDLE → (inicio + carga sel. + puerta cerrada + agua llena) → LAVADO → (automático) → CENTRIFUGADO → (automático) → SECADO → (automático) → FIN → (5 s, automático) → IDLE

Se definieron además dos condiciones de guarda adicionales a las especificadas en el enunciado, como decisiones de diseño propias: primero, el sistema ignora la pulsación de inicio si aún no se seleccionó un tipo de carga, y segundo, durante la ejecución del ciclo (lavado-centrifugado-secado-fin) el sistema no atiende nuevas pulsaciones de los botones de inicio o selección de carga, replicando el comportamiento esperable de una lavadora real, que no permite modificar la carga a mitad de ciclo.

TABLE II. ESTADOS DE LA MÁQUINA DE ESTADOS FINITOS

Estado	Descripción	LED activo	Condición de transición de salida
IDLE	Espera	Listo para lavar	Pulsador inicio

	selección de carga y arranque		+ carga seleccionada + puerta cerrada + agua llena → LAVADO
LAVADO	Motor gira 2-4s, pausa 1-3s, repite x5 (según carga)	Proceso de lavado	Automático al completar 5 repeticiones → CENTRIFUGADO
CENTRIFUGADO	Motor parpadea (simula máxima velocidad) 15-21s	Proceso de centrifugado	Automático al vencer el tiempo → SECADO
SECADO	Motor gira derecha, pausa, gira izquierda (tramos según carga)	Proceso de secado	Automático al completar los tres tramos → FIN
FIN	Ciclo terminado, motor apagado, se muestra 5s	Fin del proceso	Automático tras 5s → IDLE

2) Ajuste de tiempos según tipo de carga

Cada uno de los tres procesos ajusta su duración según la carga seleccionada, guardada en un registro de estado, siguiendo la relación indicada en el enunciado (incrementos fijos de 1 s por nivel de carga en el lavado, 3 s en el centrifugado y 2 s por tramo en el secado). Los valores resultantes se resumen en la Tabla III.

TABLE III. TIEMPOS DE PROCESO SEGÚN TIPO DE CARGA

Proceso	Carga ligera	Carga media	Carga pesada
Lavado (giro / pausa, x5)	2s / 1s	3s / 2s	4s / 3s
Centrifugado (total)	15s	18s	21s
Secado (por tramo: der. / pausa / izq.)	5s / 5s / 5s	7s / 7s / 7s	9s / 9s / 9s

3) Configuración de puertos y retardo base

Se configuraron DDRB y DDRC íntegramente como salida para los LEDs de estado y de carga respectivamente, y DDRD con los bits 6 y 7 como salida (control de motor y dirección) y el resto como entrada, habilitando el pull-up interno en los bits correspondientes a los pulsadores. La rutina de retardo base (DELAY_1MS) se calibró mediante el cálculo de ciclos de un doble lazo anidado (dec/brne), corrigiendo un primer valor estimado que introducía un error del 24% respecto al tiempo objetivo, hasta alcanzar una precisión de aproximadamente 0.03%. Sobre esta base se construyeron los retardos de mayor duración (DELAY_SEG, DELAY_MS_R20) por composición de llamadas.

4) Lectura de pulsadores y sensores

Las rutinas LEER_INICIO y LEER_SELECCION implementan antirrebote por confirmación (lectura, retardo de 25 ms, relectura) seguidas de una espera activa hasta la liberación del pulsador, de forma de generar un único evento por pulsación física. Los sensores de puerta y agua, al no requerir antirrebote por ser señales de estado continuo, se verifican en conjunto mediante la rutina VERIFICAR_LISTO, que evalúa ambos bits de PIND en una sola comparación.

5) Subrutinas de proceso

Cada uno de los tres procesos se implementó como una subrutina independiente (LAVADO, CENTRIFUGADO, SECADO), que consulta subrutinas auxiliares de tiempo (parametrizadas por el registro de carga) antes de accionar el motor. Para representar visualmente la condición de "máxima velocidad" del centrifugado, sin contar con modulación por ancho de pulso (PWM) dentro del alcance de este laboratorio, se optó por hacer parpadear el indicador de motor durante todo el proceso, mediante la subrutina BLINK_MOTOR.

B. Contador digital con tres botones y display de 7 segmentos

1) Diseño y designación de pines

Se implementó un contador decimal de 0 a 9 sobre el ATmega328P, controlado mediante tres pulsadores de incremento, decremento y reinicio. El valor del contador se visualiza permanentemente en un display de siete segmentos de cátodo común. Los pulsadores son activos en bajo y utilizan las resistencias pull-up internas del microcontrolador.

La asignación de pines utilizada para el contador se resume en la Tabla IV.

TABLE IV. MAPA DE PINES - CONTADOR Y DISPLAY DE 7 SEGMENTOS

Puerto/Pin	Función
PD2 / D2	Segmento a
PD3 / D3	Segmento b
PD4 / D4	Segmento c
PD5 / D5	Segmento d
PD6 / D6	Segmento e
PD7 / D7	Segmento f
PB0 / D8	Segmento g
PB1 / D9	Incrementar
PB2 / D10	Decrementar
PB3 / D11	Reiniciar

2) Codificación del display

Para representar los dígitos se utilizó una tabla de patrones almacenada en memoria de programa con formato 0gfdcb. Los bits correspondientes a los segmentos a-f se desplazan dos posiciones para escribirse en PD2-PD7, mientras que el segmento g se controla mediante PB0.

TABLE V. PATRONES PARA DISPLAY DE CÁTODO COMÚN

Dígito	Hex	Segmentos encendidos
0	3F	a b c d e f
1	06	b c
2	5B	a b d e g
3	4F	a b c d g
4	66	b c f g
5	6D	a c d f g
6	7D	a c d e f g
7	07	a b c
8	7F	a b c d e f g
9	6F	a b c d f g

3) Configuración de puertos GPIO

DDRD se configuró con PD2-PD7 como salidas para los segmentos a-f. En PORTB, PB0 se configuró como salida para el segmento g y PB1-PB3 como entradas para los pulsadores. Al escribir un nivel lógico alto en PORTB1-PORTB3 se habilitaron las resistencias pull-up internas, por lo que una pulsación se detecta como un nivel lógico bajo.

4) Control del contador y límites

El programa inicia el contador en 0 y comprueba continuamente los tres pulsadores con prioridad RESET, luego incremento y finalmente decremento. El incremento se limita a 9 y el decremento a 0, evitando acceder a posiciones fuera de la tabla de patrones. El pulsador RESET fuerza el valor 0 desde cualquier estado del contador.

5) Conversión y visualización del dígito

La subrutina MOSTRAR_DIGITO utiliza el registro Z para acceder a TABLA_7SEG y la instrucción LPM para leer desde memoria de programa el patrón correspondiente al valor actual. Posteriormente distribuye los bits a-f sobre PD2-PD7 y el bit g sobre PB0, conservando activos los pull-up de PB1-PB3.

6) Antirrebote y espera de liberación

Para evitar múltiples conteos producidos por el rebote mecánico de los pulsadores se implementó una rutina de antirrebote de aproximadamente 20 ms. Después de confirmar la pulsación, el programa espera la liberación del botón y vuelve a aplicar el retardo antes de regresar al ciclo principal, garantizando una única acción por pulsación.

C. Secuencias de LEDs controladas por pulsadores

1) Diseño y designación de pines

Para este ejercicio se implementó en lenguaje ensamblador un sistema capaz de ejecutar ocho secuencias diferentes sobre ocho LEDs utilizando el microcontrolador ATmega328P de la placa Arduino Uno. Los ocho LEDs se conectaron al PORTD (PD0-PD7) y los tres pulsadores de control se conectaron a PB0, PB1 y PB2.

La asignación final de pines se definió de acuerdo con la función de cada elemento del sistema.

TABLE VI. MAPA DE PINES - SECUENCIAS DE LEDs

Puerto/Pin	Función
PD0-PD7 / D0-D7	Salida para los 8 LEDs
PB0 / D8	Siguiente secuencia
PB1 / D9	Secuencia anterior
PB2 / D10	Volver a secuencia 1

2) Selección y control de secuencias

El número de secuencia se almacena en un registro de trabajo. PB0 incrementa la selección, PB1 la decrementa y PB2 restablece directamente la secuencia 1. Se incorporaron comparaciones y saltos condicionales para limitar la selección entre 1 y 8, evitando valores fuera del rango.

3) Lectura de pulsadores y cambio de secuencia

Los pulsadores se leen de forma continua y, luego de detectar una pulsación válida, el programa espera su liberación antes de aceptar un nuevo cambio. De esta forma, una única pulsación física produce un solo avance, retroceso o reinicio de la selección.

4) Ejecución de las secuencias luminosas

Cada una de las ocho secuencias ejecuta un patrón diferente sobre PORTD y se mantiene activa de forma continua hasta que el usuario selecciona otra. Para generar los patrones se utilizaron máscaras de bits, operaciones de desplazamiento y saltos, de acuerdo con lo solicitado en la consigna.

5) Retardos y actualización de los LEDs

Los cambios de estado de los LEDs se separan mediante rutinas de retardo por software. Esto permite visualizar claramente el desplazamiento o cambio de patrón y mantener una velocidad de ejecución adecuada durante la simulación y el montaje físico.

D. Unidad Aritmético-Lógica (ALU)

1) Diseño y mapa de pines

Antes de escribir el código se definió el mapa de pines a utilizar. Durante la etapa de simulación se realizaron modificaciones en la asignación inicial para adaptar correctamente las entradas, las salidas y las banderas de estado al Arduino Uno con ATmega328P. La configuración final utiliza PC0-PC3 para el operando A, PD0-PD3 para el operando B y PD4-PD6 para la señal de selección S. El resultado F se representa mediante PB0-PB3, mientras que las banderas C y N

utilizan PB4 y PB5, respectivamente, y la bandera Z se asigna a PC4/A4.

TABLE IV. MAPA DE PINES DEL ATMEGA328P

Puerto/Pin	Función
PC0-3	Entrada A (4b), pull-up
PC4 / A4	Bandera Z (Zero)
PD0-3	Entrada B (4b), pull-up
PD4-6	Entrada S (3b), pull-up
PB0-3	Resultado F (4b)
PB4	Bandera C (Carry)
PB5	Bandera N (Negativo)

2) Tabla de operaciones

La ALU implementa las ocho operaciones especificadas en la consigna, resumidas en la Tabla V junto con el comportamiento esperado de la bandera de Carry para cada una.

TABLE V. TABLA DE OPERACIONES DE LA ALU

S2S1 S0	Operación	F	Carry
000	CLEAR	00	0
001	A - B	A - B	préstamo
010	A + B	A + B	acarreo
011	XOR	A XOR B	0
100	AND	A AND B	0
101	OR	A OR B	0
110	SHL	A << 1	acarreo
111	INC	A + 1	acarreo

3) Configuración de puertos y lectura de entradas

En la configuración final, PB0-PB5 se establecieron como salidas, PC0-PC3 como entradas y PC4 como salida, mientras que PORTD se configuró como entrada. Los pines de entrada mantienen habilitadas las resistencias pull-up internas. La subrutina LEER_ENTRADAS lee directamente A desde PC0-PC3 y B desde PD0-PD3, sin aplicar la instrucción COM. Para obtener el selector S se enmascaran PD4-PD6 mediante ANDI y se realizan cuatro desplazamientos LSR, de modo que PD4, PD5 y PD6 queden alineados como S0, S1 y S2 en los bits bajos del registro rS.

4) Decodificación y ejecución de las operaciones

La subrutina OPERAR decodifica el valor de S mediante una cadena de comparaciones (CPI/BREQ) que salta a la rutina correspondiente a cada una de las ocho operaciones. Se optó por esta estructura, en lugar de una tabla de saltos indexada, por ser más directa de justificar y de verificar operación por operación durante la puesta a punto.

5) Cálculo de banderas de estado

La bandera C se determina observando el bit 4 del resultado antes de enmascararlo a 4 bits. La bandera N se obtiene a partir del bit 3 del resultado F, que corresponde al bit más significativo del resultado de 4 bits. La bandera Z se activa cuando el resultado F es igual a 0000. En la implementación final, C se visualiza en PB4/D12, N en PB5/D13 y Z en PC4/A4.

6) Actualización de salidas

La subrutina ACTUALIZAR_SALIDAS combina el resultado F, ubicado en PB0-PB3, con las banderas C y N, ubicadas en PB4 y PB5. La bandera Z se actualiza de forma independiente mediante PC4/A4. De esta manera se elimina el uso de PB6 para la bandera Z y también la visualización del selector S mediante PC4-PC6. El ciclo principal repite continuamente la lectura de entradas, la ejecución de la operación, la actualización de las salidas y el retardo de refresco.

6. RESULTADOS

A. Lavadora Automática

El sistema fue verificado de forma incremental en el simulador Proteus. Se validó individualmente el funcionamiento de la configuración de puertos, la calibración de los retardos, la detección de pulsaciones sin bloqueos del programa ante fallas de conexión del circuito, la rotación correcta de los tres niveles de carga, y las condiciones de guarda de inicio (carga seleccionada, puerta cerrada y agua llena). Durante la integración se detectaron y corrigieron los siguientes defectos: un error de temporización en el parpadeo del centrifugado que reducía su duración a la mitad del valor especificado, y una condición no contemplada que permitía iniciar el ciclo sin haber seleccionado previamente un tipo de carga. Tras aplicar las correcciones correspondientes, el sistema ejecuta correctamente la secuencia completa IDLE→LAVADO→CENTRIFUGADO→SECADO→FIN→IDLE, con los tiempos ajustados según el tipo de carga seleccionada.

B. Contador digital con tres botones y display de 7 segmentos

Durante la verificación se comprobó que el display inicia mostrando 0, que cada pulsación válida de incremento o decremento modifica el contador una sola unidad y que RESET restablece inmediatamente el valor a 0. También se verificó la saturación en los límites: al incrementar desde 9 el valor permanece en 9 y al decrementar desde 0 permanece en 0. La rutina de antirrebote y la espera de liberación evitaron conteos múltiples ante pulsaciones prolongadas.

C. Secuencias de LEDs controladas por pulsadores

Durante la simulación se comprobó el funcionamiento de las ocho secuencias luminosas y de los tres pulsadores de control. PB0 permitió avanzar a la secuencia siguiente, PB1 regresar a la anterior y PB2 volver directamente a la secuencia 1. También se verificó que la selección permaneciera limitada entre las secuencias 1 y 8 y que cada patrón se ejecutara de forma continua hasta recibir una nueva orden. Posteriormente, el montaje físico con Arduino Uno reprodujo correctamente el comportamiento observado en la simulación.

D. Unidad Aritmético-Lógica (ALU)

La verificación se realizó de forma incremental en PICSimLab. Primero se comprobó la configuración de los puertos y la lectura directa de los operandos A y B; luego se verificó la correcta decodificación del selector S mediante PD4-PD6. Finalmente se probaron individualmente las ocho operaciones de la ALU (CLEAR, SUB, ADD, XOR, AND, OR, SHL e INC) y se comprobó la correspondencia entre el resultado F y las banderas C, N y Z. Durante esta etapa se corrigieron la inversión de las entradas, la lectura del selector y la asignación física de la bandera Z.

7. CONCLUSIONES

A. Lavadora Automática

El desarrollo incremental, verificando cada bloque funcional antes de integrarlo a la máquina de estados completa, resultó fundamental para aislar y corregir errores que hubieran sido considerablemente más difíciles de detectar en el sistema integrado (en particular, el error de temporización en el centrifugado, que no era evidente sin una medición explícita de tiempos). Asimismo, la práctica evidenció la importancia de diseñar las rutinas de lectura de entradas de forma que nunca bloqueen indefinidamente el flujo del programa, dado que un pulsador o sensor en estado inesperado no debe impedir la operación del resto del sistema.

B. Contador digital con tres botones y display de 7 segmentos

El desarrollo del contador permitió aplicar el control directo de puertos GPIO, el uso de tablas almacenadas en memoria de programa y la lectura de pulsadores activos en bajo. La separación del programa en rutinas para lectura, antirrebote y visualización facilitó la implementación y verificación. El control de los límites entre 0 y 9 y la espera de liberación de los pulsadores garantizaron que el display recibiera siempre un patrón válido y que cada pulsación generara una única acción.

C. Secuencias de LEDs controladas por pulsadores

El ejercicio permitió aplicar el manejo de puertos digitales, máscaras de bits, desplazamientos, saltos condicionales y rutinas de retardo en lenguaje ensamblador. La utilización de pulsadores para seleccionar las ocho secuencias permitió comprobar la importancia de controlar los límites de selección y esperar la liberación de cada pulsador para evitar cambios múltiples. La comparación entre simulación y montaje físico confirmó el funcionamiento del programa implementado.

D. Unidad Aritmético-Lógica (ALU)

La puesta a punto de la ALU permitió comprobar la importancia de adaptar el software al mapa real de pines utilizado en la simulación y en el montaje físico. Las principales modificaciones consistieron en corregir la lectura de A y B eliminando la inversión mediante COM, sustituir el procesamiento original del selector S por cuatro desplazamientos LSR y reorganizar las salidas para ubicar F en PB0-PB3, C en PB4, N en PB5 y Z en PC4/A4. Luego de estos ajustes se verificó correctamente el funcionamiento de las ocho operaciones y de las tres banderas de estado.

BIBLIOGRAFIA

- [1] Microchip Technology Inc., *ATmega328/P Datasheet*, documento DS40002061.
- [2] Microchip Technology Inc., *AVR Instruction Set Manual*, documento DS40002198.
- [3] Arduino, *Arduino Uno Rev3 – Technical Specifications*.
- [4] Tecnologías de Microprocesamiento, *Laboratorio 1 – GPIO*, material de práctica, 2026.
- [5] Archivos complementarios del Laboratorio N.º 1: códigos fuente, simulaciones y material de implementación. Disponible en Google Drive: Carpeta de Google Drive

<https://drive.google.com/drive/folders/1xDx-frTT12cXEP4mP5czVe1nG1ynLyCd>